# Manual Técnico – Sistema de Gestión de Alquiler de Vehículos

Proyecto: SistemaGestionAlquilerVehiculos

(versión 1.0-SNAPSHOT)

Curso: Programación II

Universidad Mariano Gálvez de Guatemala

Grupo: Los Depuradores

Repositorio: github.com/WON-2008jim/SistemaGestionAlquilerVehiculos

Fecha del manual: 5 de noviembre de 2026

## 1. Introducción

Este documento explica cómo está hecho el Sistema de Gestión de Alquiler de Vehículos, una aplicación de escritorio en Java que desarrollamos en el grupo Los Depuradores para el curso de Programación II. La idea del programa es llevar el control de un negocio de alquiler: quiénes son los clientes, qué empleados trabajan en cada sucursal, qué vehículos hay y en qué estado están, cuándo entran a mantenimiento, qué contratos se firman y qué facturas se emiten.

El manual está pensado para quien tenga que darle mantenimiento al proyecto o seguir desarrollándolo. Por eso, además de describir las clases y la base de datos, incluye los pasos para instalarlo y una lista honesta de lo que todavía está pendiente o mal resuelto.

### 1.1 Cómo funciona a grandes rasgos

La persona usuaria inicia sesión con el usuario y la clave de un empleado y entra a un menú principal desde el que abre una ventana por cada módulo. Los datos se guardan en una base de datos PostgreSQL a la que el programa se conecta mediante JDBC. Cada ventana no habla directamente con la base de datos: llama a una clase "gestor", que es la que arma las consultas, las ejecuta y devuelve el resultado. Además, desde las ventanas de contratos y de facturas se pueden generar comprobantes en PDF con la biblioteca iText.

## 2. Requisitos y tecnologías

El proyecto está escrito en Java y se administra con Apache Maven; el desarrollo se hizo en Apache NetBeans, y de ahí vienen los archivos .form que describen las ventanas. El README pide JDK 17 o superior. La base de datos es PostgreSQL; el respaldo del repositorio se generó con la versión 18.6. La conexión se hace con el controlador org.postgresql:postgresql 42.6.0 y los reportes PDF con com.itextpdf:itextpdf 5.5.13.3. La interfaz es Java Swing y el código se versiona con Git en GitHub.

## 3. Estructura del proyecto

```
SistemaGestionAlquilerVehiculos/
├── pom.xml                      Maven: JDBC de PostgreSQL e iText
├── nbactions.xml                Acciones de NetBeans (ejecuta vista.login)
├── README.md / CHANGELOG.md
├── alquilerVehiculosScript.sql  Volcado de la base (formato custom de pg_dump)
├── Diagrama UML Proyecto.drawio Diagrama de clases
├── baseDatos/
│   └── alquilerVehiculos.backup Respaldo de la base (formato custom de pg_dump)
├── docs/
│   └── Manual de Usuario.pdf    Manual para quienes usan el sistema
├── Documentacion Final/         Documento técnico final (PDF)
└── src/main/
    ├── java/
    │   ├── conexion/            ConexionBD
    │   ├── logica/              GestorClientes, GestorVehiculos, GestorSucursales,
    │   │                        GestorEmpleados, GestorContratos, GestorFacturas,
    │   │                        GestorMantenimientos
    │   ├── vista/               login, MenuPrincipal y las ventanas de cada módulo
    │   └── com/mycompany/sistemagestionalquilervehiculos/
    │                            las 10 clases del modelo
    └── resources/
        ├── icons/               Iconos de botones y menús
        └── media/               logo.png
```

En el paquete vista hay una ventana por módulo: VentanaClientes, VentanaVehiculos, VentanaSucursales, VentanaEmpleados, VentanaContratos, VentanaMantenimiento y VentanaFacturas, cada una con su .java y su .form. La opción Manual de Usuario del menú Ayuda busca el archivo con la ruta relativa docs/Manual de Usuario.pdf, así que el programa debe ejecutarse desde la carpeta raíz del proyecto para encontrarlo.

## 4. Arquitectura

El programa está dividido en cuatro capas, cada una en su propio paquete, para que la interfaz no tenga que saber cómo se guardan los datos.

La **capa de modelo** (paquete com.mycompany.sistemagestionalquilervehiculos) reúne las clases de entidad: objetos simples con atributos privados, constructores, getters y setters, un toString() y, en algunos casos, métodos con reglas de negocio.

La **capa de vista** (paquete vista) contiene los formularios Swing hechos con el editor de NetBeans. Estas clases leen lo que escribe la persona, llaman a los gestores y muestran los resultados en tablas.

La **capa de lógica y acceso a datos** (paquete logica) tiene los gestores. Cada uno implementa el esquema alta, búsqueda, cambio y baja (ABCD) más un listado, siempre con PreparedStatement. Varios incluyen además métodos cargarTabla... que llenan directamente el modelo de tabla de una ventana, y un método privado mapear... que convierte una fila del resultado en un objeto del modelo.

La **capa de conexión** (paquete conexion) es una sola clase, ConexionBD, que usa el patrón Singleton para que todo el programa comparta la misma conexión JDBC.

El recorrido normal de una operación es este: la ventana llama al gestor, el gestor pide la conexión a ConexionBD.getInstancia().getConexion(), prepara la consulta con un PreparedStatement y la ejecuta en PostgreSQL.

## 5. Base de datos

La base se llama alquilerVehiculos y tiene diez tablas en el esquema public. Todas las llaves primarias son enteros con identidad generada automáticamente (GENERATED ALWAYS AS IDENTITY), de modo que el programa nunca debe indicar el id al insertar.

### 5.1 El respaldo y el script de la base de datos

El repositorio trae dos archivos con la base de datos: baseDatos/alquilerVehiculos.backup y alquilerVehiculosScript.sql. Los dos están en el formato custom de pg_dump y los dos incluyen ya las columnas que el código necesita: empleado.telefono y empleado.id_sucursal, y en mantenimiento id_sucursal, id_mecanico, fecha_estimada y estado. El respaldo se volvió a generar en las últimas actualizaciones del proyecto, porque la versión del 14 de octubre todavía no tenía esas columnas. Las tablas que siguen se describen tal como quedaron en esta versión.

### 5.2 Catálogos: categorías y sucursales

categoria_vehiculo guarda el nombre de la categoría (único y obligatorio), una descripción opcional y la tarifa base, que debe ser mayor que cero. sucursal guarda el nombre, la dirección y el teléfono, todos obligatorios.

### 5.3 Personas: clientes y empleados

Cada cliente tiene nombre, DPI (obligatorio y único), NIT, teléfono y correo opcionales, y el número de licencia, que es obligatorio porque sin licencia no se puede alquilar. Cada empleado tiene nombre, rol, usuario (único) y clave, todos obligatorios, un teléfono opcional y la sucursal a la que pertenece (id_sucursal, obligatoria y con llave foránea hacia sucursal).

### 5.4 Vehículos y mantenimientos

vehiculo guarda marca, modelo, VIN y placa (los dos últimos únicos), el año (desde 2000), el kilometraje (no negativo), la tarifa diaria (mayor que cero) y el estado, que por defecto es Disponible. Cada vehículo pertenece a una categoría y a una sucursal, ambas obligatorias.

mantenimiento guarda la fecha de ingreso (fecha_ingreso), el tipo de servicio, el costo (no negativo) y el vehículo atendido; si se borra el vehículo, sus mantenimientos se borran en cascada. También guarda la sucursal donde se hace el trabajo (id_sucursal), el mecánico responsable (id_mecanico, que es un empleado), la fecha estimada de entrega (fecha_estimada) y el estado, que por defecto es Ingresado y pasa a Finalizado cuando termina el trabajo. La columna descripcion sigue existiendo como dato opcional, pero el código actual no la usa.

### 5.5 Contratos y facturas

contrato_alquiler tiene fecha de inicio y fin, días de alquiler (mayor que cero), monto base y total (no negativos) y un estado que por defecto es Activo. Siempre referencia a un cliente, un vehículo y el

empleado que lo registró. Los cargos adicionales de un contrato van en detalle_contrato (descripción del cargo y monto), que se borra en cascada junto con el contrato.

factura tiene un número de serie único, la fecha de emisión, el total y el contrato al que corresponde. Sus renglones están en detalle_factura (concepto y monto), también con borrado en cascada.

### 5.6 Relaciones

Una categoría y una sucursal agrupan muchos vehículos. Un vehículo puede tener muchos mantenimientos. Un contrato pertenece a un cliente, a un vehículo y a un empleado, puede tener varios cargos adicionales y puede generar facturas, y cada factura tiene sus renglones de detalle.

## 6. Descripción de las clases

### 6.1 Conexión

ConexionBD es un Singleton. getInstancia() devuelve la única instancia y la vuelve a crear si la conexión es nula o está cerrada; getConexion() entrega el objeto java.sql.Connection para ejecutar consultas; cerrarConexion() la cierra y libera la instancia. El constructor es privado y carga el driver org.postgresql.Driver. Los datos de conexión están escritos como constantes dentro de la clase.

### 6.2 Modelo

Las diez clases del modelo corresponden una a una con las tablas de la base de datos: CategoriaVehiculo, Sucursal, Cliente, Empleado, Vehiculo, Mantenimiento, ContratoAlquiler, DetalleContrato, Factura y DetalleFactura. Cada una tiene sus atributos, constructores, getters, setters y toString(). Los métodos de negocio más relevantes son:

- Cliente: validarLicencia() devuelve verdadero si el número de licencia no es nulo ni vacío.
- ContratoAlquiler: asociarCliente() solo asocia al cliente si su licencia es válida; también tiene asociarVehiculo(), agregarDetalle(), calcularDias(), calcularTotal() y finalizarContrato().
- Vehiculo: registrarKilometraje() rechaza un valor menor al kilometraje actual, cambiarEstado() modifica el estado y consultarDisponibilidad() es verdadero cuando el estado es Disponible (sin importar mayúsculas).
- Mantenimiento: registrarMantenimiento() pone el vehículo en estado En mantenimiento y finalizarMantenimiento() lo regresa a Disponible.
- Factura: generarFactura() y calcularTotal(), que toma el total del contrato asociado, además de imprimirFactura().
- Empleado: autenticar() y obtenerPerfil(). Sucursal y CategoriaVehiculo tienen métodos para registrar, actualizar y consultar sus datos.

En Cliente, Empleado, Sucursal y Vehiculo el toString() devuelve un texto corto y legible (el nombre, o marca, modelo y placa) para poder mostrar el objeto directamente en listas desplegables de la interfaz.

### 6.3 Gestores

Todos los gestores guardan en PostgreSQL con PreparedStatement.

GestorClientes permite agregar, buscar por DPI o por id, modificar (por DPI, solo teléfono y correo, o actualizar el registro completo), eliminar por DPI y listar. Rechaza un DPI vacío o que ya esté registrado.

GestorVehiculos tiene las mismas operaciones, pero buscando y eliminando por placa. Maneja los estados Disponible, Ocupado y En mantenimiento.

GestorSucursales ofrece alta, búsqueda por id, actualización, eliminación y listado.

GestorEmpleados incluye autenticar(usuario, clave), que compara ambos datos dentro de la consulta SQL y devuelve el Empleado o null. La clave se guarda y se compara en texto plano.

GestorContratos es el más completo: agrega, busca por id o por cliente, modifica, cambia el estado, renueva, elimina, lista y calcula estadísticas. Antes de guardar valida que existan el cliente, el vehículo y el empleado y que las fechas, los días y el monto tengan sentido. Al crear un contrato marca el vehículo como Ocupado, y al finalizarlo o eliminarlo lo regresa a Disponible.

GestorFacturas agrega, busca (por número de serie, por id o por contrato), modifica, elimina por número de serie y lista. Al agregar valida que el número de serie no esté vacío ni repetido y que la factura tenga un contrato asociado; si no se indicó el total toma el del contrato, y si no hay fecha usa la de hoy.

GestorMantenimientos registra un mantenimiento recibiendo datos sueltos (sucursal, vehículo, mecánico, tipo de servicio, fechas, costo y estado) en lugar de un objeto Mantenimiento. Al registrarlo pone el vehículo en En mantenimiento y al finalizarlo lo regresa a Disponible, todo dentro de una transacción.

### 6.4 Vistas y punto de entrada

El programa arranca en la clase vista.login, que es la que ejecuta nbactions.xml. Pide usuario y clave, los valida con GestorEmpleados.autenticar() y, si son correctos, abre MenuPrincipal.

MenuPrincipal tiene una barra de menús con cuatro opciones: Archivo (cerrar sesión y salir), Gestión (clientes, vehículos y categorías, sucursales, empleados), Operaciones (contratos de alquiler, mantenimientos y facturación) y Ayuda (abre el Manual de Usuario en PDF de la carpeta docs). Cada entrada abre su ventana.

VentanaClientes, VentanaVehiculos, VentanaSucursales y VentanaEmpleados son formularios con una tabla de registros; VentanaClientes, por ejemplo, tiene los botones Registrar, Modificar, Eliminar y Limpiar. VentanaContratos registra, finaliza, renueva y elimina contratos: calcula el total como días por tarifa diaria, verifica que el vehículo esté Disponible y, con el botón Reporte, genera un comprobante en PDF con iText. VentanaMantenimiento sirve para registrar y dar seguimiento a los mantenimientos usando GestorMantenimientos.

VentanaFacturas es el panel de emisión de facturas. La persona elige un contrato de una lista desplegable y la ventana muestra la fecha de hoy, el cliente, el vehículo (por su id) y el total del contrato. El número de serie se propone solo, con el formato FAC- seguido de un número, y se puede cambiar. Al presionar Confirmar y generar factura se registra la factura con GestorFacturas y se actualiza el historial de facturas emitidas. Si se selecciona una factura de esa tabla, el botón Imprimir Reporte

pide dónde guardar el archivo y genera su comprobante en PDF con iText; Limpiar Datos deja el formulario listo para otra factura.

Los iconos están en src/main/resources/icons y el logo en media/logo.png.

## 7. Instalación y configuración

### 7.1 Antes de empezar

Hay que instalar un JDK (ver la sección 9 sobre qué versión usar), Apache Maven, Apache NetBeans y PostgreSQL, y comprobar que el servicio de PostgreSQL esté activo en el puerto 5432.

### 7.2 Crear la base de datos

Primero se crea la base vacía:

```
createdb -U postgres alquilerVehiculos
```

Después se restaura uno de los dos archivos del repositorio. Ambos están en el formato custom de pg_dump, así que se restauran con pg_restore (o con la opción Restore de pgAdmin) y no con psql, aunque alquilerVehiculosScript.sql tenga extensión .sql. Con el respaldo de la carpeta baseDatos el comando es:

```
pg_restore -U postgres -d alquilerVehiculos baseDatos/alquilerVehiculos.backup
```

También se puede restaurar desde pgAdmin con clic derecho sobre la base de datos, opción Restore, y seleccionando el archivo. Para el respaldo conviene usar una versión de PostgreSQL igual o superior a la que lo generó.

### 7.3 Configurar la conexión

En conexion/ConexionBD.java hay que editar las constantes URL, USUARIO y CONTRASENA para que coincidan con el servidor local de cada quien.

### 7.4 Descargar y compilar

```
git clone https://github.com/WON-2008jim/SistemaGestionAlquilerVehiculos.git
cd SistemaGestionAlquilerVehiculos
mvn clean compile
```

Maven descarga solo el controlador JDBC 42.6.0 y la biblioteca iText 5.5.13.3.

### 7.5 Ejecutar

Se abre el proyecto en NetBeans y se ejecuta la clase vista.login (clic derecho, Run File, o la acción Run del proyecto, porque nbactions.xml ya apunta a ella). Aparece la pantalla de inicio de sesión; se ingresan el usuario y la clave de un empleado que exista en la tabla empleado y, si son correctos, se abre el menú principal.

## 8. Historial de cambios

- 2026-10-05: estructura inicial del proyecto Maven, .gitignore, las 10 clases del modelo, README y CHANGELOG; métodos de CategoriaVehiculo, Cliente, ContratoAlquiler y DetalleContrato.

- 2026-10-07: métodos de Sucursal, Vehiculo, Mantenimiento, Factura, DetalleFactura y Empleado.
- 2026-10-14: dependencia JDBC, ConexionBD, base de datos con su respaldo y GestorClientes con CRUD.
- 2026-10-22: GestorMantenimientos y GestorVehiculos.
- 2026-10-26: GestorContratos y GestorFacturas con persistencia (según el CHANGELOG).
- 2026-11-02: formulario de login, carpeta resources, paquete vista y mejoras en los gestores.
- 2026-11-05: formularios del módulo Gestión, GestorSucursales y mejoras en los gestores.
- 2026-11-05: mejoras en gestores y clases, cambios en la base de datos, iText en el pom.xml y formularios del módulo Operaciones.
- 2026-11-05: ventana de facturación con emisión y reporte en PDF, apertura del Manual de Usuario desde el menú Ayuda, respaldo de la base actualizado, script de la base de datos, diagrama UML y documentación final (commits be351a2 y a492bff).

## 9. Glosario

- **DPI:** Documento Personal de Identificación, el documento de identidad en Guatemala.
- **NIT:** Número de Identificación Tributaria.
- **CRUD / ABCD:** altas, búsquedas, cambios y bajas de registros.
- **JDBC:** la interfaz de Java para conectarse a bases de datos.
- **Singleton:** patrón de diseño que asegura que de una clase exista una sola instancia.
- **POJO:** clase Java sencilla, con atributos, getters y setters.